Awantika AIparc 3

March 30, 2026

[7]:
```python
#Prim's Algorithm

import heapq

def prim(graph, start):
    visited = set([start])
    edges = []
    min_heap = []

    for neighbor, weight in graph[start]:
        heapq.heappush(min_heap, (weight, start, neighbor))

    while min_heap:
        weight, u, v = heapq.heappop(min_heap)

        if v not in visited:
            visited.add(v)
            edges.append((u, v, weight))

            for neighbor, w in graph[v]:
                if neighbor not in visited:
                    heapq.heappush(min_heap, (w, v, neighbor))

    return edges




n = int(input("Enter number of vertices: "))
e = int(input("Enter number of edges: "))

graph = {i: [] for i in range(n)}

print("Enter edges (u v weight):")
for _ in range(e):
    u, v, w = map(int, input().split())
    graph[u].append((v, w))
```

```
    graph[v].append((u, w))  # undirected graph

start = int(input("Enter starting vertex: "))


mst = prim(graph, start)
print("Prim's MST:")
for u, v, w in mst:
    print(f"{u} - {v} : {w}")
```

```
Enter number of vertices:  5
Enter number of edges:  6

Enter edges (u v weight):

 0 1 2
 0 3 6
 1 2 3
 1 3 8
  1 4 5
 2 4 7
Enter starting vertex:  0

Prim's MST:
0 - 1 : 2
1 - 2 : 3
1 - 4 : 5
0 - 3 : 6
```

[8]:
```
#Kruskal's Algorithm

def find(parent, x):
    if parent[x] != x:
        parent[x] = find(parent, parent[x])
    return parent[x]


def union(parent, rank, x, y):
    rootX = find(parent, x)
    rootY = find(parent, y)

    if rootX != rootY:
        if rank[rootX] < rank[rootY]:
            parent[rootX] = rootY
        else:
            parent[rootY] = rootX

        if rank[rootX] == rank[rootY]:
            rank[rootX] += 1
```

```python
def kruskal(vertices, edges):
    edges.sort(key=lambda x: x[2])

    parent = {v: v for v in vertices}
    rank = {v: 0 for v in vertices}

    mst = []

    for u, v, weight in edges:
        if find(parent, u) != find(parent, v):
            union(parent, rank, u, v)
            mst.append((u, v, weight))

    return mst



n = int(input("Enter number of vertices: "))
e = int(input("Enter number of edges: "))

vertices = list(range(n))
edges = []

print("Enter edges (u v weight):")
for _ in range(e):
    u, v, w = map(int, input().split())
    edges.append((u, v, w))


mst = kruskal(vertices, edges)

print("Kruskal's MST:")
for u, v, w in mst:
    print(f"{u} - {v} : {w}")
```

```
Enter number of vertices:  5
Enter number of edges:  6

Enter edges (u v weight):

 0 1 2
 0 3 6
 1 2 3
 1 3 8
 1 4 5
 2 4 7
```

```
Kruskal's MST:
0 - 1 : 2
1 - 2 : 3
1 - 4 : 5
0 - 3 : 6
```